

Análise de Código Java

Token Bucket — Controlo de Taxa de Requisições

OBJETIVO GERAL

A classe **TokenBucket** implementa o algoritmo *Token Bucket*, amplamente utilizado para controlo de taxa de requisições (*rate limiting*). O seu objetivo é limitar o número de operações permitidas num dado período de tempo, garantindo que um sistema não seja sobrecarregado por rajadas de pedidos.

LÓGICA E ALGORITMO

O balde começa cheio com **capacity** tokens. Cada chamada a **consume(n)** verifica primeiro se há tokens suficientes — se sim, subtrai-os e retorna **true**; caso contrário, retorna **false** sem bloquear. O método privado **refill()** reabastece o balde proporcionalmente ao tempo decorrido desde o último acesso, usando **System.nanoTime()** para precisão de nanossegundos, com um teto em **capacity** via **Math.min()**.

CONCEITOS JAVA UTILIZADOS

O método **consume()** é declarado **synchronized**, garantindo segurança em ambientes multi-thread (*thread-safety*) sem necessidade de locks explícitos. São usados tipos primitivos **double** para precisão de vírgula flutuante no cálculo de tokens fracionários, e **long** para o timestamp em nanossegundos. O método **main()** demonstra o uso com **Thread.sleep(500)**, simulando intervalos de 500 ms entre requisições.

COMPLEXIDADE E DESEMPENHO

Cada operação **consume()** tem complexidade **O(1)** em tempo e **O(1)** em espaço — nenhuma estrutura de dados cresce com o número de requisições. O estado é mantido em apenas quatro campos escalares, tornando esta implementação extremamente eficiente e adequada para cenários de alta frequência.

DECISÕES DE DESIGN E LIMITAÇÕES

O design é intencionalmente não-bloqueante: requisições rejeitadas falham imediatamente em vez de aguardar tokens disponíveis, o que é ideal para APIs que devem responder com baixa latência. A principal limitação é que o estado é **in-memory** e não persistente — em ambientes distribuídos com múltiplas instâncias, seria necessário um armazenamento partilhado (como Redis) para coordenação global do limite de taxa.

SAÍDA DO MÉTODO main() (EXEMPLO)

```
Request 0: true    // 10 tokens → consome 3 → fica com 7
Request 1: true    // +1 token (500ms × 2.0) → 8 → consome 3 → fica com 5
Request 2: true    // +1 token → 6 → consome 3 → fica com 3
Request 3: true    // +1 token → 4 → consome 3 → fica com 1
Request 4: false   // +1 token → 2 → insuficiente para 3
Request 5: true    // +1 token → 3 → consome 3 → fica com 0
```